

```
1 #include "gen_table.h"
2 #include "linked_list.h"
3
4 gentabl_t* create_empty_gentabl(int n) {
5     gentabl_t* gt = malloc(sizeof(gentabl_t));
6     gt->num_elts = 0;
7     gt->size = n;
8
9     gt->decomps = malloc(gt->size*sizeof(list_t*));
10    for (int i = 0; i < gt->size; i++) {
11        gt->decomps[i] = create_empty_list();
12    }
13    return gt;
14 }
15
16 bool is_empty_gentabl(gentabl_t* gt) {
17     assert(gt != NULL);
18     return (gt->size == 0);
19 }
20
21 void clear_gentabl(gentabl_t* gt) {
22     for (int i = 0; i < gt->size; i++) {
23         free_list(gt->decomps[i]);
24     }
25     free(gt->decomps);
26     free(gt);
27 }
28
29 bool is_empty_index(gentabl_t* gt, int index) {
30     assert(index < gt->size);
31     list_t* to_check = gt->decomps[index];
32     assert(to_check != NULL);
33     return (is_empty(to_check));
34 }
35
36 list_t* get_copy_index_val(gentabl_t* gt, int i) {
37     assert(!is_empty_index(gt, i));
38     list_t* copy = deep_copy(gt->decomps[i]);
39     return copy;
40 }
41
42 void copy_at_index(gentabl_t* gt, int index, list_t* val) {
43     assert(gt != NULL);
44     assert(index < gt->size);
45     if (gt->decomps[index] != NULL) {
46         free_list(gt->decomps[index]);
47     }
48
49     gt->decomps[index] = deep_copy(val);
50 }
51
52 void del_index(gentabl_t* gt, int index) {
53     assert(gt != NULL);
54     assert(index < gt->size);
55     list_t* to_free = gt->decomps[index];
56     free_list(to_free);
57     gt->decomps[index] = create_empty_list();
58 }
59
60 void show_all_gentabl(gentabl_t* gt) {
```

```
61     for (int i = 0; i < gt->size; i++) {
62         printf("[%d]: ", i);
63         show_values(gt->decomps[i]);
64         printf("\n");
65     }
66 }
67
68 void free_gentabl(gentabl_t* gt) {
69     assert(gt != NULL);
70     for (int i = 0; i < gt->size; i++) {
71         free_list(gt->decomps[i]);
72     }
73     free(gt->decomps);
74     free(gt);
75 }
```